1. Git vs GitHub

Interview Question

What is the difference between Git and GitHub?

Answer

Many people think Git and GitHub are the same, but they are completely different.

Git	GitHub
Version Control System	Cloud Repository Hosting Platform
Installed on Local Machine	Runs on Cloud
Tracks code changes	Stores Git repositories online
Open Source Tool	Web-based Service
Works Offline	Requires Internet (mostly)
Created by Linus Torvalds	Owned by Microsoft

Simple Example

Imagine you are writing a book.

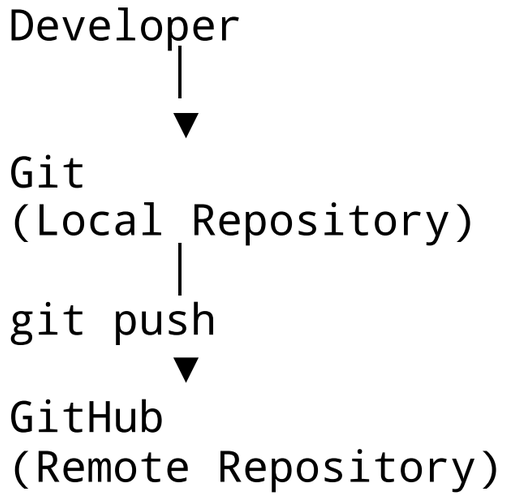
Git

- Keeps track of every chapter you write.
- Allows you to go back to any previous version.
- Works on your laptop.

GitHub

- Stores your book online.
- Allows multiple authors to collaborate.
- Acts as a backup.
- Enables Pull Requests and Code Reviews.

Workflow



Common Git Commands

```
git init
git add .
git commit -m "Initial Commit"
git status
git log
```

Common GitHub Operations

```
git push origin main
git pull origin main
git clone <repo-url>
```

Interview Tip

Git = Tool

GitHub = Service

2. Difference between Git Fetch and Git Pull

This is one of the most frequently asked DevOps interview questions.

Git Fetch

```
git fetch origin
```

What happens?

Downloads the latest changes from GitHub.

BUT

Does NOT merge them.

Diagram

```
GitHub
   |
git fetch
   |
Local Repository

Working Directory

No Changes
```

Example

Developer A pushes code.

You execute

```
git fetch
```

Now you know there are new commits.

But your project files remain unchanged.

Git Pull

```
git pull origin main
```

What happens?

Downloads

-

Automatically merges

Diagram

```
GitHub
   |
```

```
git fetch
    |
git merge
    |
Working Directory Updated
```

Interview Answer

```
git pull = git fetch + git merge
```

Difference Table

Git Fetch	Git Pull
Downloads latest changes	Downloads + Merges
Safe	May create conflicts
Doesn't modify working directory	Updates working directory
Used before reviewing changes	Used when ready to update

Best Practice

Production:

```
git fetch
git log
git diff
git merge
```

Not

```
git pull
```

immediately.

3. Merge vs Rebase

Another favorite interview question.

Merge

Example

Branch

main

```
A-----B-----C
           \
            D-----E
```

Merge

git merge feature

Result

```
A-----B-----C-----------M
           \           /
            D---------E
```

Notice

Merge Commit

M

is created.

Advantages

- Preserves history
- Easy to understand
- Safe
- Recommended for shared branches

Rebase

git rebase main

Result

```
A----B----C----D'----E'
```

Looks like feature branch started from latest main.

No merge commit.

Cleaner history.

Difference

Merge	Rebase
Creates Merge Commit	No Merge Commit
Preserves Branch History	Rewrites History
Safe	Dangerous on Shared Branch
Best for Teams	Best for Local Branches

Interview Tip

Never rebase a branch that other developers are already using.

4. How do you resolve Merge Conflicts?

Suppose

Developer A changes

```
System.out.println("Hello");
```

Developer B changes

```
System.out.println("Welcome");
```

Git doesn't know which one to keep.

Conflict occurs.

Conflict looks like

```
<<<<<<< HEAD

Hello
```

```
=======

Welcome

>>>>>>> feature
```

Steps

Step 1

```
git pull
```

Conflict appears.

Step 2

Open file.

Choose correct code.

Example

```
System.out.println("Hello Welcome");
```

Step 3

```
git add .
```

Step 4

```
git commit
```

Step 5

```
git push
```

Conflict resolved.

Interview Answer

First I identify conflicting files using Git status, manually resolve the conflicting code, verify the application builds successfully, stage the resolved files using `git add`, commit the merge resolution, and then push the changes after testing.

5. You pushed a bad commit to Production.

What will you do?

This is one of the most important DevOps questions.

Situation

You pushed

```
Commit A

Commit B

Commit C (Bad)
```

Production breaks.

Option 1 (Recommended)

Git Revert

```
git revert <commit-id>
```

Example

```
git revert 3a4bc9
```

What happens?

Creates a new commit that undoes the bad commit.

History remains intact.

Result

A

B

C

D (Undo C)

Safe.

Option 2

If commit is NOT pushed

Use

```
git reset --soft HEAD~1
```

or

```
git reset --hard HEAD~1
```

Difference

Soft

Commit removed

Changes remain

Hard

Commit removed

Changes removed

Why not use Reset in Production?

Suppose

10 developers already pulled the bad commit.

If you execute

```
git reset --hard
git push --force
```

Everyone's history changes.

Chaos.

Production Best Practice

Always use

```
git revert
```

Real Interview Answer

If the bad commit is already deployed to production and shared with the team, I would use `git revert` to create a new commit that safely reverses the changes without rewriting Git history. After reverting, I would validate the application in a staging environment if possible, redeploy the fixed version through the CI/CD pipeline, and monitor the application. I avoid `git reset --hard` and force pushes on shared branches because they rewrite history and can disrupt other developers.

Interview Cheat Sheet

Question	Best Answer
Git vs GitHub	Git is a Version Control System, GitHub is a cloud repository hosting platform.
Fetch vs Pull	Fetch downloads changes only; Pull downloads and merges changes.
Merge vs Rebase	Merge preserves history with a merge commit; Rebase creates a cleaner linear history by rewriting commits.
Merge Conflict	Resolve conflicts manually, test the code, stage changes, commit, and push.
Bad Commit in Production	Use `git revert` for shared/production branches. Avoid `git reset --hard` and force pushes.

⭐ Bonus Interview Question

Interviewer: *"Why shouldn't we use `git push --force` on the main branch?"*

Answer:

Because `git push --force` rewrites the commit history on the remote repository. Other developers who have already pulled the previous history may face conflicts, lost commits, or failed merges. On shared branches like `main` or `production`, the safer approach is to use `git revert`, which preserves history while undoing the unwanted changes.